

软件系统设计设计模式考前问卷

1. Every Design Pattern allows the developer to vary different parts of a design. Complete the following table:

Purpose	Design Pattern	Aspect(s) That Can Vary
creational	Abstract Factory	
structural	Adapter	
structural	Decorator	
behavioral	Observer	
behavioral	State	
creational	Factory Method	
behavioral	Strategy	
structural	Facade	
behavioral	Command	

每个设计模式都允许开发人员在设计中变化不同的部分。完成以下表格

目的	设计模式	可变化的方面
创建型	抽象工厂模式	增加新的工厂和产品族容易，增加新的产品等级困难
结构型	适配器模式	需要包装适配的接口
结构型	装饰者模式	拓展原有对象的行为和责任
行为型	观察者模式	观察者及其依赖关系
行为型	状态模式	对开闭原则支持效果不好，增加状态类或修改状态类的行为都要修改代码
创建型	工厂方法模式	添加新的工厂或产品
行为型	策略模式	策略算法或行为
结构型	外观模式	增加新的子系统
行为型	命令模式	增加新的命令请求和操作

2. In object-oriented programming one is advised to avoid case (and if) statements. Select one design pattern that helps avoid case statements and explain how it helps.

在面向对象编程中，建议避免使用 case（和 if）语句。选择一个能帮助避免使用 case 语句的设计模式，并解释它如何起作用。

以策略模式为例。策略模式定义了一系列算法，分别封装起来，并使它们可以替换。可以根据需要选择选择使用哪种策略或算法的实例，通过不同策略的选择来替换分支语句。

使用策略模式，将每个算法或行为都封装在独立的策略类中，这些策略类实现了一个共同的接口或基类。客户端代码不再需要自己判断条件并执行不同的逻辑，而是将具体的策略对象注入或设置到上下文类中。上下文类负责协调不同策略对象的使用，根据需要调用相应的策略对象的方法。这样，客户端代码只需通过调用上下文类的方法来完成相应的操作，而不需要直接处理条件逻辑。

3. The ATM provides two types of transaction, withdrawals and deposits. For every transaction, the user specifies a reference account (for deposit to or withdrawal from) and a dollar amount. Withdrawals can either be made in cash to the user or as a transfer to another account. For cash withdrawals, the user chooses the denominations of the payment; for transfers, the user specifies a target account. Draw the design class diagram. Code is not required.

ATM 提供两种类型的交易，取款和存款。对于每笔交易，用户需要指定参考账户（用于存款或取款）和金额。取款可以以现金形式支付给用户，也可以作为转账到另一个账户。对于现金取款，用户可以选择支付的面额；对于转账，用户需要指定目标账户。请绘制设计类图。不需要提供代码。

采用策略模式和装饰模式，策略模式用来选择存钱还是取钱，装饰模式用来拓展取钱方式。

